Towards a Novel Vertical Scaling Approach for Bursty Workloads in Kubernetes

Miguel de Lima[1], Luan Teylo[2],
Lúcia Drummond[1]
Federal Fluminense University[1], INRIA Bordeaux[2]

14th International Workshop on Cloud and Edge Computing, and Applications Management

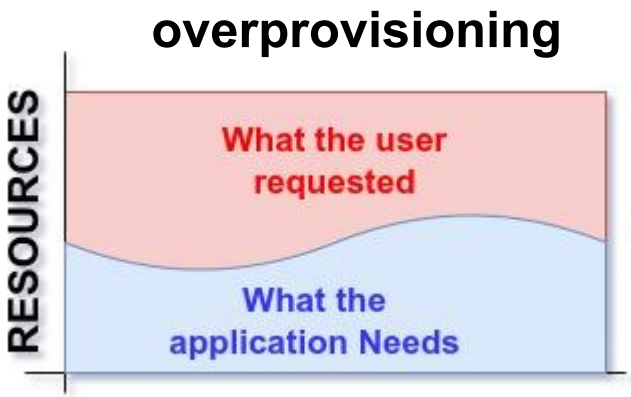
Outline

- Introduction
- Dynamic Resizing Strategy (DRS)
- Methodology & Results
- Conclusions & Future Works

Introduction

Introduction

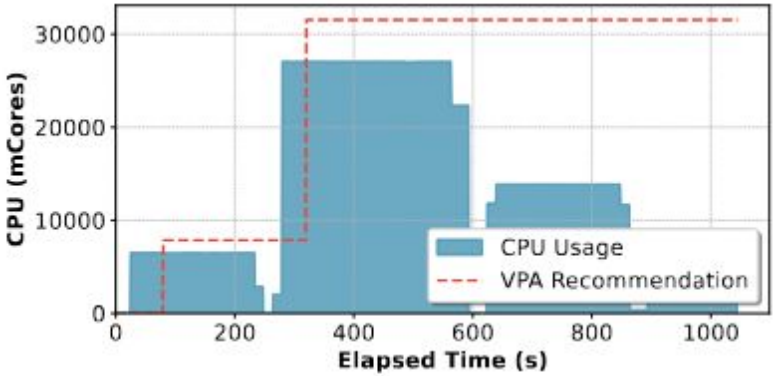
- Users often lack precise knowledge of resource needs, leading to:
  - Wasted of Resources
  - Higher costs (in the cloud)
  - Longer waiting times (in shared environments)

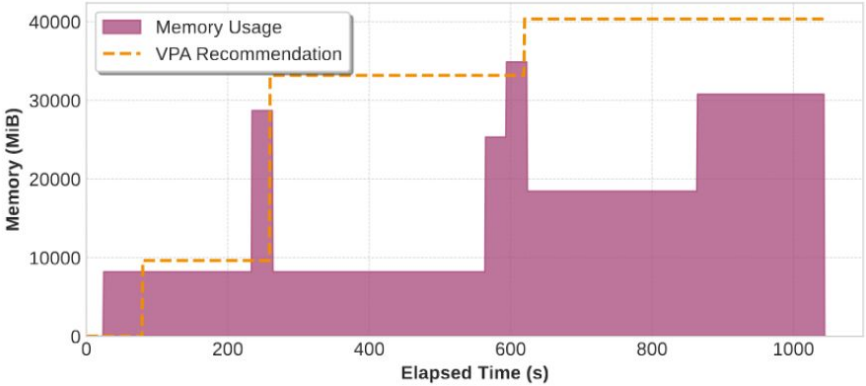


Introduction

- Allocation of Resources are usually static
  - Ex: same amount of memory is allocated from the beginning to the end
- The behaviour of application is not constant
  - Ex: sometimes it can use more memory sometimes it can use less

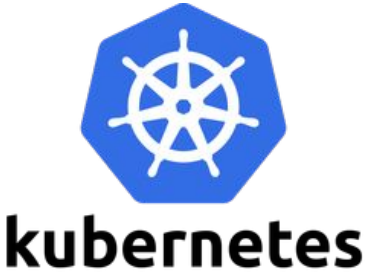
BURSTY WORKLOADS





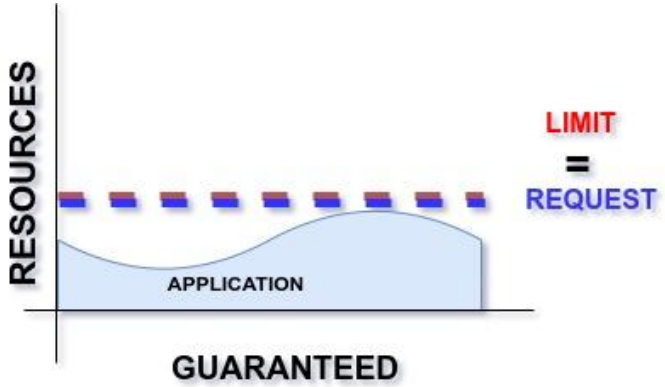
Introduction

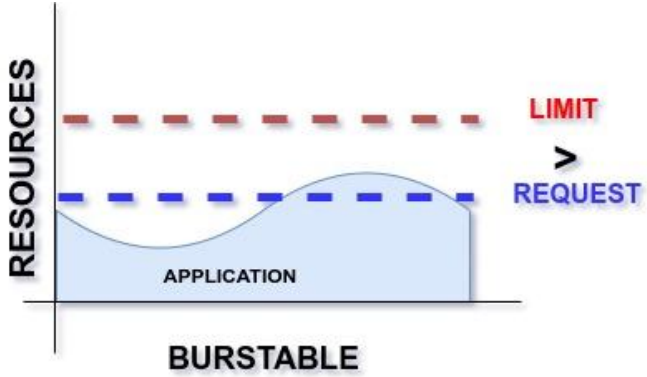
- Option to run applications in the cloud and on-premises: Containers!
- It is a orchestrator that offers flexibility in allocating resources.
  - **Request:** the amount of resources guaranteed to the application
  - **Limit:** the maximum amount of resources the application can use



Introduction

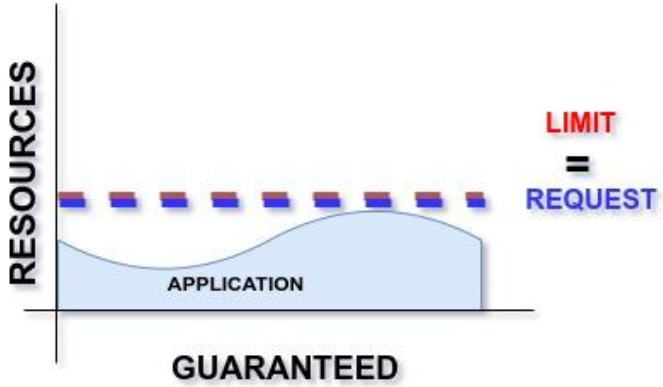
- The values of request and limit defines the QoS Classes
  - Guaranteed: request = limit. Provides predictable performance but leads to underutilization.
  - Burstable: request < limit. Increases utilization but offers weaker performance guarantees.

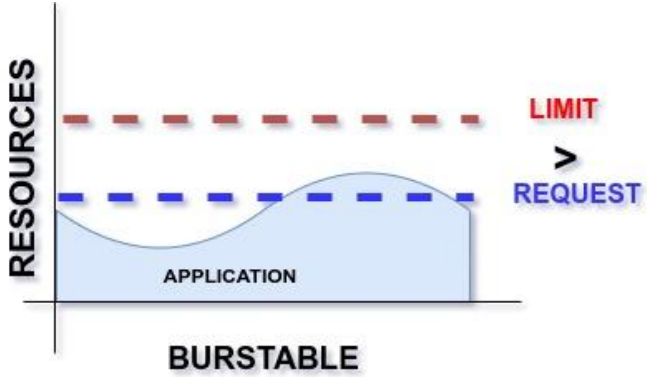




Introduction

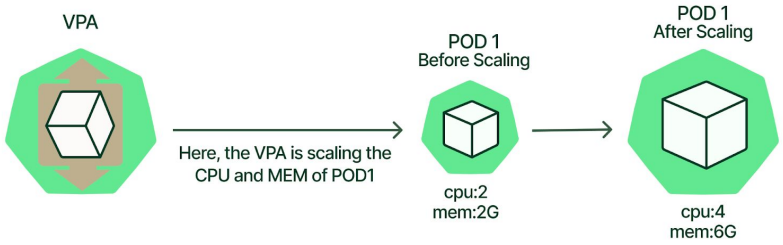
How these classes affect the performance of the applications in a shared environment?





Introduction

- Kubernetes also supports autoscaling: it adapts allocated computational resources at runtime
- Traditionally, the Vertical Pod Autoscaler requires restarting the application
  - problem for long running jobs without checkpoint
- New feature (v1.33): in-place autoscaling that **DOES NOT** require restarting the application.



Introduction

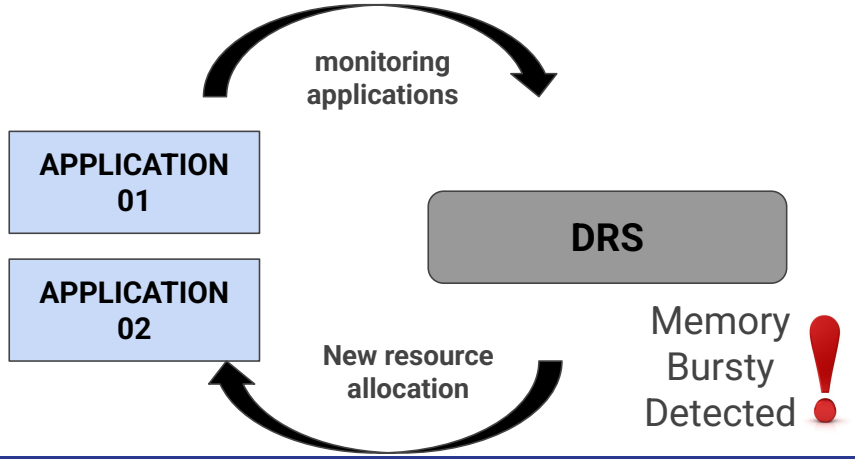
- Kubernetes also supports autoscaling: it adapts allocated computational resources at runtime
- Traditionally, the Vertical Pod Autoscaler requires restarting the application
  - problem for long running jobs without checkpoint
- New feature (v1.33): in-place autoscaling that **DOES NOT** require restarting the application.

Kubernetes doesn't provide an automatic mechanism to decide when resources should be changed!

Dynamic Resizing Strategy

Dynamic Resizing Strategy

- Use the in-place feature to avoid overprovisioning
- It minimizes the impact on the application performance in a shared environment
- Adapt the resources allocated to the applications according to their usage

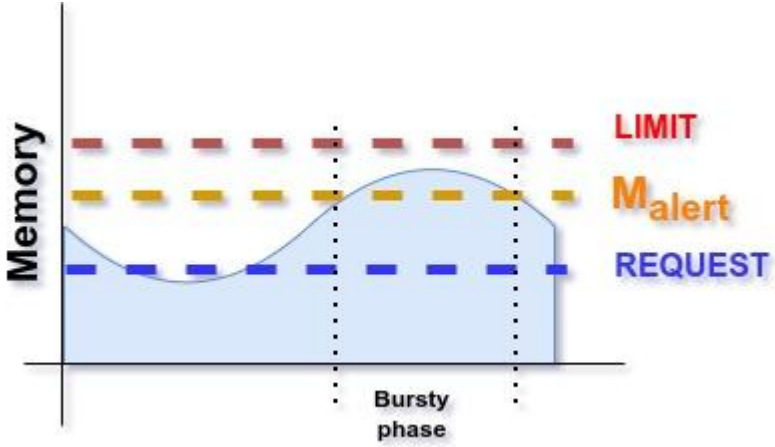


Dynamic Resizing Strategy

The memory threshold:

DRS assumes the application profile is known, with:

- $\mathbf{M_{request}}$: *request* value
- $\mathbf{M_{limit}}$: *limit* value



M_{alert} is calculated dynamically:

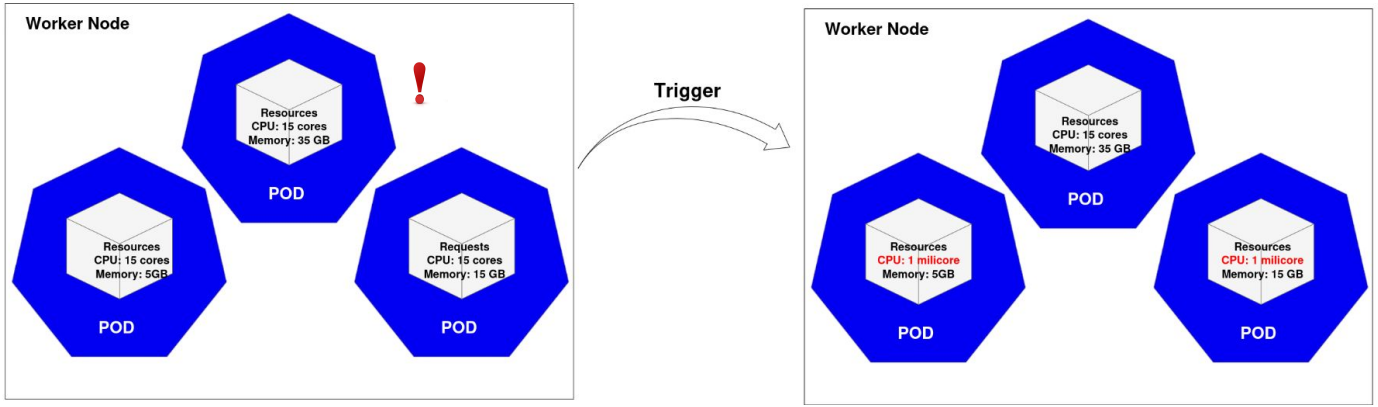
$$M_{\text{alert}} = M_{\text{request}} + (M_{\text{limit}} - M_{\text{request}}) \times F_{\text{trigger}}$$

$\mathbf{F_{trigger}}$ is the trigger factor (set to 0.5 in our experiments).

Dynamic Resizing Strategy

The Preventive Quarantine Mechanism:

- When memory usage of a an application bigger than M_{alert}:
  - DRS slow down all the other applications by settings their CPU requests and limits to a minimal value (1 millicore) using in-place scaling.
  - The system exits quarantine, and throttled resources are restored once the memory usage of the prioritized application drops below M_{alert}



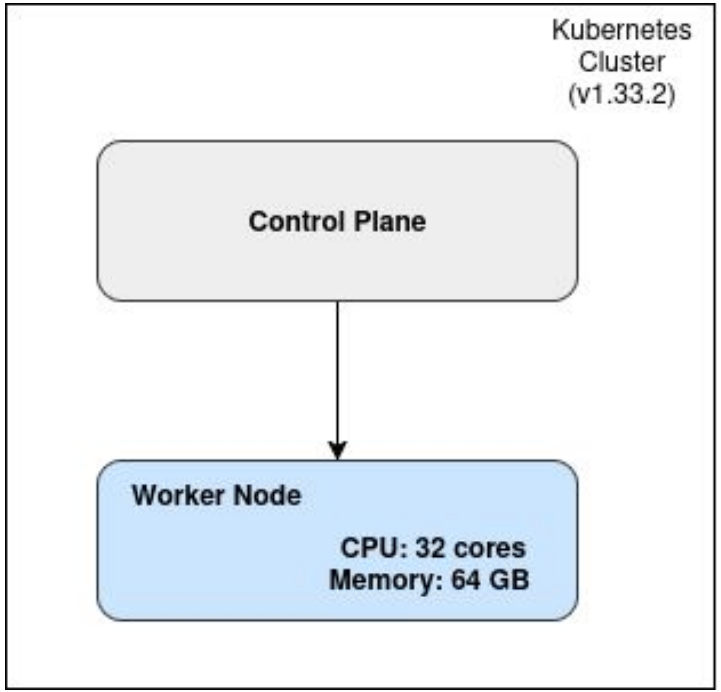
Methodology & Results

Experimental Methodology

Environment

- Kubernetes cluster (v1.33.2)
- Two Nodes: control plane and a worker node
- Worker node with 32 CPU cores and 64 GB of RAM

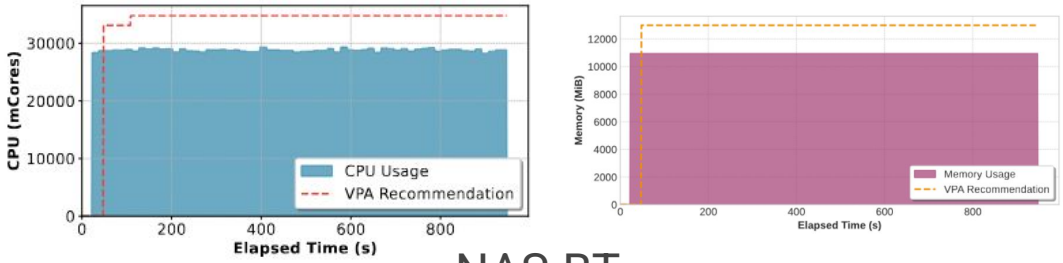
Goal: Compare standard Kubernetes QoS vs. DRS in a shared environment



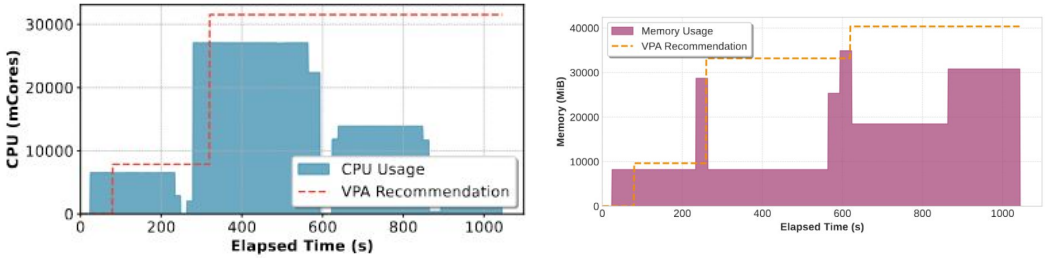
Experimental Methodology

Workloads

- **NAS Parallel Benchmarks (NPB):** classic benchmark for HPC (represents steady-state applications)
- **Synthetic Applications:** Suite built on a C++ driver to model behaviors with significant CPU and memory variability



NAS-BT



SIM-SPIKES

Profiling & Baseline Characterization

- Applications were profiled individually to characterize dynamic demands.
- Data showed high heterogeneity
  - Some apps had stable profiles (NAS-BT)
  - Others had substantial differences between average and peak memory consumption (SIM-Epochs)

Application	CPU Usage (mCores)		Memory Usage (MiB)	
	Average	Peak	Average	Peak
NAS-BT	28,836	29,713	10,958	10,958
NAS-CG	27,675	29,316	16,741	16,742
NAS-EP	31,023	31,899	34	35
NAS-MG	28,223	28,473	27,149	27,149
NAS-SP	28,532	29,136	11,469	11,469
SIM-Epochs	14,541	25,016	16,391	45,145
SIM-Spikes	13,855	27,114	16,484	34,885
SIM-Sawtooth	17,355	26,367	30,943	57,457
SIM-Staircase	17,253	25,302	24,762	33,348

NAS applications have stable behaviour, while SIM applications have higher variability

Co-execution Scenarios

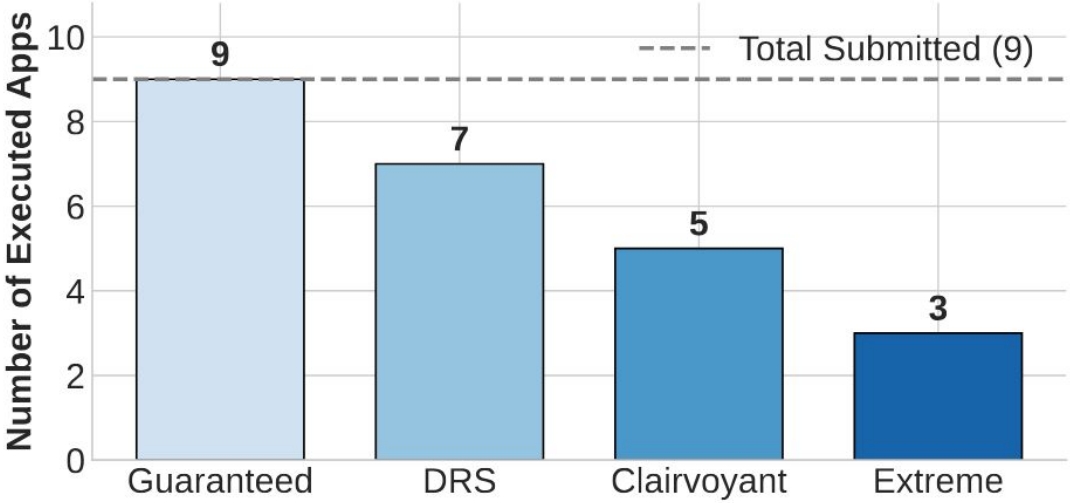
Guaranteed (Non Bursting)	request = limit = Peak utilization.
Clairvoyant (Burstable)	request = Average Usage limit = peak utilization
Extreme (Burstable)	requests = minimal resource limits = maximum available resource (32 CPU, 62.5 Gi Memory).
Dynamic Resizing Strategy	Dynamically Values

Result

System Stability

Metric:

Number of applications that successfully completed (Application that doesn't fail of OUT).



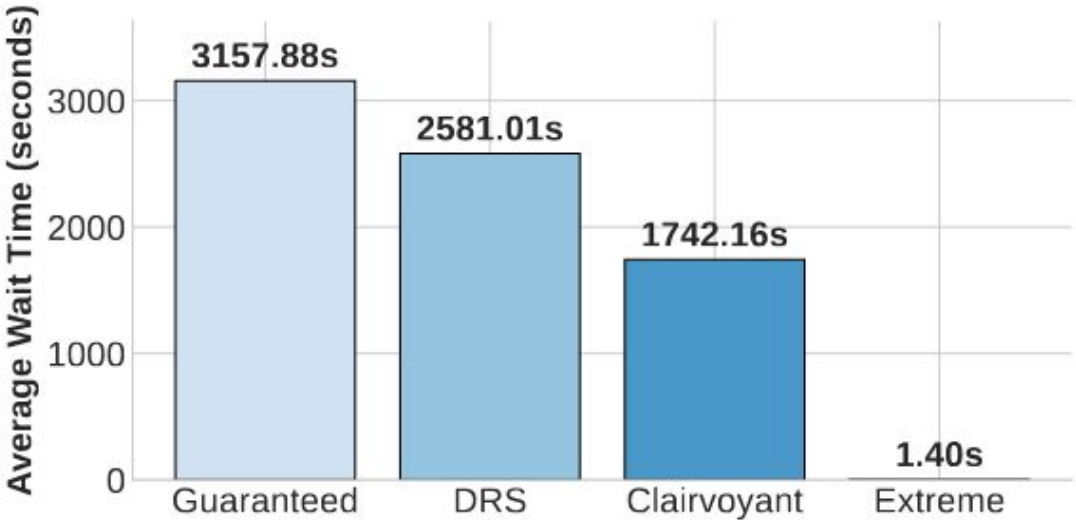
DRS is a robust alternative, approaching the stability of the Guaranteed strategy

Result

Scheduling Efficiency

Metric:

Average time applications spent waiting to be scheduled.



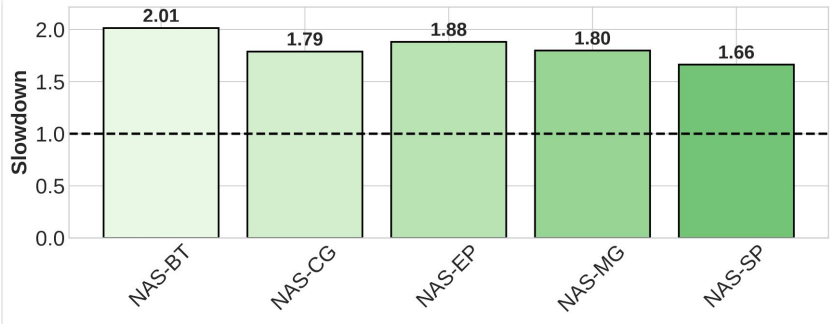
DRS effectively balances stability with much faster pod placement. Reduced waiting time in 18%

Result

Performance Degradation

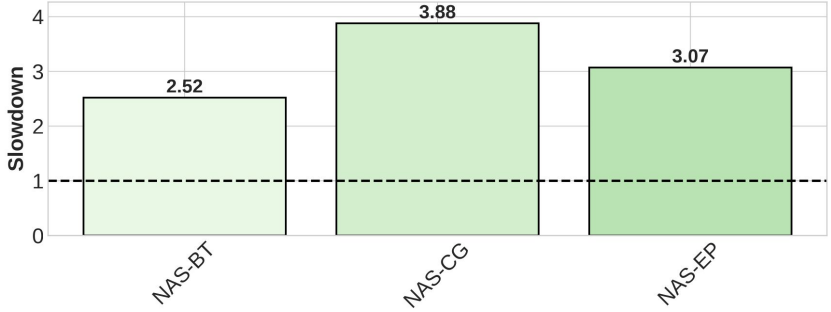
Metric:

$$slowDown = \frac{COExecution_time}{Isolation_time}$$



Clairvoyant

Clairvoyant/Extreme: Severe and uncontrolled degradation for surviving applications (up to 1.8x and nearly 4.0x, respectively).



Extreme

Result

Performance Degradation

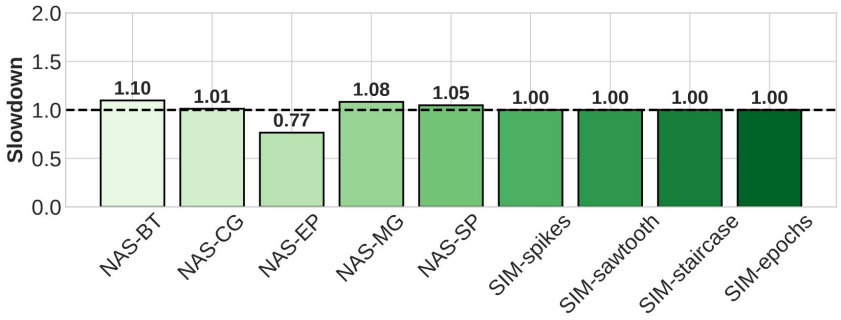
Metric:

$$slowDown = \frac{CO Execution_time}{Isolation_time}$$

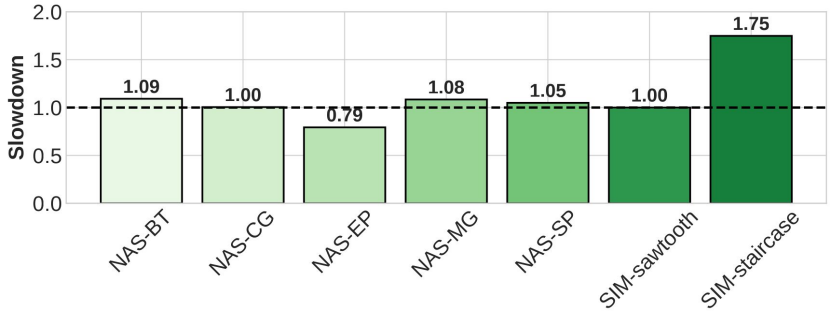
Guaranteed: Slowdown near 1.0 (Strong performance isolation).

DRS: Introduced a measurable but controlled cost (Slowdown up to 1.7x).

DRS prevents failures while mitigating the worst-case performance penalties inherent in overcommitment.



Guaranteed



DRS

Conclusion & Future Work

Conclusion
Navigating the Trade-Off

- Native QoS strategies force a difficult trade-off
  - **Guaranteed** provides stability at the cost of high latency
  - **Burstable** (Clairvoyant/Extreme) prioritizes speed at the expense of reliability.
- DRS effectively navigating this trade-off.
  - achieves a high success rate
  - reducing queue times by over 18%.

Takeaway: An adaptive approach like DRS provides an effective path for running dynamic, bursty workloads on Kubernetes.

Future Work

- Improve DRS reliability under high event rates.
- Extend experiments to **multi-node** deployments with heterogeneous nodes.
- Study scheduler interactions and explicitly consider **CPU contention** alongside memory pressure.
- Incorporate **monetary cost factors** to optimize throughput while minimizing cloud expenses.

Thank you, Obrigado, Merci

Towards a Novel Vertical Scaling Approach for Bursty Workloads in Kubernetes

Miguel de Lima[1], Luan Teylo[2],
Lúcia Drummond[1]
Federal Fluminense University[1], INRIA Bordeaux[2]

Questions?

If you prefer, contact me: miguel_flj@id.uff.br

14th International Workshop on Cloud and Edge Computing, and Applications Management